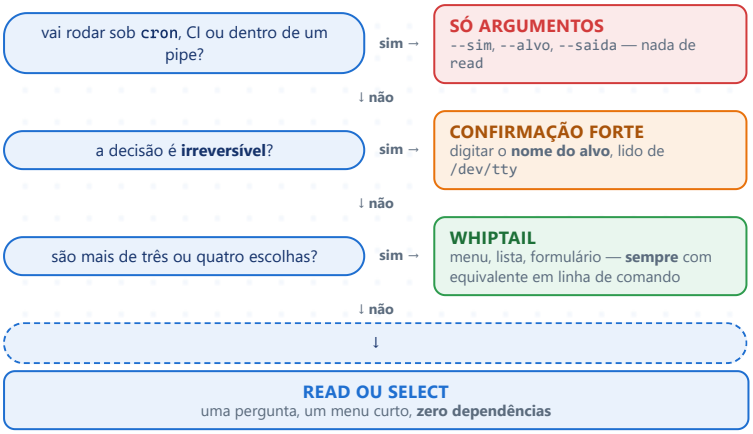
GUIA VISUAL 076

INTERAÇÃO: MENUS, CORES E CAIXAS DE DIÁLOGO

076
GUIAS LINUX

*O script de limpeza era cuidadoso: mostrava o tamanho e pedia `read -p "Confirma? [s/N] "` antes do `rm -rf`. Funcionou por meses — até alguém escrever `gerar_lista.sh | limpar.sh`. O `read` leu a **primeira linha da lista** em vez do teclado. Era `srv-web-01`, não bateu com `s`, e foi só sorte. **O defeito não é o `read`: é supor que existe alguém do outro lado.** Script bem escrito verifica se há interação e se comporta de forma segura quando não há.*

QUE INTERFACE USAR · E A ANATOMIA DA COR



ANATOMIA DE UM ESCAPE ANSI

`\e`	`[`	`1;31`	`m`
ESC (0x1b) inicia a sequência	CSI abre os parâmetros	atributos: 1 = negrito, 31 = vermelho	SGR aplica os atributos

`0` normal · `1` negrito · `4` sublinhado
`31` vermelho · `32` verde · `33` amarelo
`34` azul · `36` ciano · `37` branco · `41`–`47` as mesmas no fundo
`\e[0m` **sempre** no fim — sem ele a cor vaza para o resto do terminal

Prefira `tput` a escrever a sequência à mão. `tput setaf 1`, `tput bold` e `tput sgr0` consultam o terminfo do terminal em uso e devolvem **string vazia** quando ele não suporta cor — a proteção acontece sozinha. Escrever `\e[1;31m` direto funciona em "quase" todo lugar, e esse "quase" inclui o arquivo de log, o `cron` e o terminal de quem usa leitor de tela.

A pergunta do topo da árvore é a que mais economiza retrabalho: se o script pode acabar num pipe ou num agendador — e quase todo script acaba —, a interatividade tem de ser **opcional**. Escreva primeiro a versão com argumentos e só depois acrescente o modo interativo que preenche os mesmos valores; o contrário, começar pelo menu e tentar automatizar depois, custa uma reescrita. **As três guardas antes de emitir cor** são o destino ser um terminal (`[[ -t 1 ]]`), a variável `NO_COLOR` não estar definida e o `TERM` não ser `dumb`. Se qualquer uma falhar, as variáveis de cor viram string vazia e **o resto do script não muda em nada** — a cor é opcional em **tempo de execução**, não em tempo de escrita. O mesmo `[[ -t 1 ]]` guarda a barra de progresso: sem ele, um log de `cron` acumula milhares de linhas de `\r`. Para progresso de um **fluxo de dados**, o `pv` já faz tudo isso sem uma linha de código.

READ, SELECT E WHIPTAIL NA PRÁTICA

kali@kali: ~/lab-interacao

```
# leia do TERMINAL, não do stdin — e falhe alto se não houver
$ read -rp 'Confirma? [s/N] ' r < /dev/tty
$ interativo() { [[ -t 0 && -t 1 ]]; }
# sem terminal: RECUSE, não assuma. --sim libera o modo automático
$ printf 's\n' | ./seguro.sh; echo "status: $?"
erro: exige confirmacao; use --sim para modo nao interativo
status: 1
# o irreversível não aceita "s": exige o nome por extenso
$ read -rp "Digite o nome do alvo para confirmar: " c < /dev/tty
$ [[ "$c" == "$ALVO" ]] || { echo 'cancelado.' >&2; exit 1; }
# cor sob guarda tripla: desligada, tudo vira string vazia
$ if [[ -t 1 && -z "${NO_COLOR:-}" && "${TERM:-dumb}" != dumb ]]; then
    C_RESET=$(tput sgr0); C_VERM=$(tput setaf 1)
  else C_RESET='' C_VERM=''; fi
$ erro() { printf '%s[ERRO ]%s %s\n' "$C_VERM" "$C_RESET" "$*" >&2; }
# menu sem dependência nenhuma; case sobre $REPLY, não sobre o texto
$ PS3='Escolha uma opção: '
$ select o in "Listar" "Reiniciar" "Sair"; do case "$REPLY" in …
# whiptail desenha no stdout e devolve pelo stderr: troque os três
$ escolha=$(whiptail --menu "Tarefa:" 15 60 4 \
      "1" "Atualizar" "2" "Limpar" 3>&1 1>&2 2>&3); echo $?
0   # 0 confirmou · 1 cancelou · 255 pressionou Esc
```

READ: AS OPÇÕES QUE IMPORTAM

`-r`	não interpreta a barra invertida — **sempre**
`-p 'txt'`	prompt antes de ler; sai no `stderr`, para não poluir o `stdout` que outro programa pode consumir
`-s`	não ecoa o digitado (senha)
`-n N`	retorna após N caracteres, sem esperar Enter
`-t N`	desiste após N segundos, status > 128 — **obrigatório** em prompt que possa travar script automatizado
`-e -i val`	edição de linha com valor inicial editável
`-a arr` · `-u N`	lê para um array · lê do descritor N

WHIPTAIL: AS CAIXAS

`--yesno`	pergunta; status **0** sim, **1** não, **255** Esc — tratar os três é o que evita que um Esc vire confirmação
`--inputbox`	uma linha de texto · `--passwordbox` sem eco
`--checklist`	múltipla escolha com `ON`/`OFF`
`--gauge`	barra alimentada por percentuais na entrada
`--stdout`	dispensa a ginástica `3>&1 1>&2 2>&3` nas versões recentes

O CHECKLIST DA INTERAÇÃO

- ✓ prompt e diagnóstico em `stderr`, **resultado em `stdout`** — é o que faz `r=$(script.sh)` funcionar
- ✓ confirmação lida de `/dev/tty`, nunca do `stdin`
- ✓ **Enter vazio nunca confirma** operação destrutiva
- ✓ modo não interativo **explícito** (`--sim`, `--batch`) e recusa clara sem terminal
- ✓ toda pergunta tem equivalente em **opção de linha de comando**

A "confirmação fantasma" é o modo mais silencioso de disparar um comando destrutivo. Num pipe, num `cron` ou num contêiner sem TTY, `read -p "Confirma? [s/N]"` não espera ninguém: consome a próxima linha do `stdin`. Se ela for `s`, o `rm -rf`, o `mkfs` ou o `cryptsetup luksFormat` rodam **sem que ninguém tenha digitado nada**. As defesas: ler de `< /dev/tty` (que **falha** quando